Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_regression import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, AdaBoostRegressor, ExtraTreesRegressor, BaggingRegressor, VotingRegressor, StackingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import cross_val_score, GridSearchCV, RandomizedSearchCV
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin
from scipy.stats import uniform, randint
```

Setting Working Directory

```
# Set working directory (adjust path as needed)
import os
print("Current working directory:", os.getcwd())
```

```
Current working directory: /kaggle/working
```

Loading the Data

```
# Load the data
df = pd.read_csv('Tesla.csv')
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist)
```

```
Dataset shape: (1257, 7)
Columns: [Date, Open, High, Low, Close, Adj Close, Volume]
```

```
df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2019-12-31	84.900	86.820	84.342	86.052	86.052	47765000
1	2020-01-02	86.052	86.084	77.607	80.536	80.536	142981500
2	2020-01-03	80.670	81.772	76.034	77.270	77.270	148244000
3	2020-01-06	77.000	79.268	75.799	77.404	77.404	147478500
4	2020-01-07	77.335	80.188	77.038	79.268	79.268	114004500

```
df.info()
```

```
<class pandas.DataFrame>
RangeIndex: 1257 entries, 0 to 1256
Data columns (total 7 columns):
 #   Date      1257 non-null   object
 #   Open      1257 non-null   float64
 #   High      1257 non-null   float64
 #   Low       1257 non-null   float64
 #   Close     1257 non-null   float64
 #   Adj Close 1257 non-null   float64
 #   Volume    1257 non-null   int64
dtype: float64, int64, object
memory usage: 69.1 KB
```

```
df.tail()
```

	Date	Open	High	Low	Close	Adj Close	Volume
1252	2024-12-23	417.860	418.000	403.000	403.000	403.000	71468000
1253	2024-12-24	403.000	410.000	402.000	403.000	403.000	32168300
1254	2024-12-26	403.000	403.000	391.000	394.000	394.000	61707000
1255	2024-12-27	394.000	394.000	379.000	379.000	379.000	62390000
1256	2024-12-30	379.000	379.000	354.000	357.000	357.000	99000000

```
# Check for missing values
print("Missing values:")
df.isnull.sum()
```

```
Missing values:
Date      0
Open      0
High      0
Low       0
Close     0
Adj Close 0
Volume    0
dtype: int64
```

```
# Basic statistics
df.describe()
```

	Open	High	Low	Close	Adj Close	Volume
count	1257.000000	1257.000000	1257.000000	1257.000000	1257.000000	1.257000e+03
mean	177.681702	181.267165	173.929753	177.681702	177.681702	7.116354e+07
std	101.544756	102.592709	100.391688	101.544756	101.544756	4.526588e+07
min	34.250000	35.398000	33.674000	34.250000	34.250000	1.185000e+07
25%	63.000000	64.000000	61.800000	63.000000	63.000000	4.140000e+07
50%	177.000000	179.000000	174.000000	177.000000	177.000000	6.000000e+07
75%	251.000000	256.000000	245.000000	251.000000	251.000000	8.800000e+07
max	479.860000	488.540000	470.817000	479.860000	479.860000	4.761000e+08

Step 2: Feature Engineering

```
# Make a copy for safety
df = df.copy()
print("Original columns:", df.columns.tolist)
print("Data types:")
df.dtypes
```

```
Original columns: [Date, Open, High, Low, Close, Adj Close, Volume]
Data types:
Date      object
Open     float64
High     float64
Low      float64
Close    float64
Adj Close float64
Volume    int64
dtype: object
```

```
# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])
print("Date range:", df['Date'].min() to df['Date'].max())
```

```
Date range: 2019-12-31 00:00:00 to 2024-12-30 00:00:00
```

```
# Create basic features
df['High_Low_Diff'] = df['High'] - df['Low']
df['Open_Close_Diff'] = df['Open'] - df['Close']
df['Volume_Millions'] = df['Volume'] / 1000000
df['Price_Range_Pct'] = (df['High_Low_Diff'] / df['Close']) * 100
df['Daily_Return'] = df['Close'].pct_change
df['Volatility_5'] = df['Daily_Return'].rolling(5).std
df['Volatility_10'] = df['Daily_Return'].rolling(10).std
df['Volatility_20'] = df['Daily_Return'].rolling(20).std
df['Volume_Ratio'] = df['Volume'] / df['Volume'].rolling(20).mean
df['Price_Position'] = (df['Close'] - df['Low']) / (df['High'] - df['Low'])
df['Gap'] = df['Open'] - df['Close'].shift(1)
df['Gap_Pct'] = (df['Gap'] / df['Close'].shift(1)) * 100
df['Upper_Shadow'] = df['High'] - df[['Open', 'Close']].max(axis=1)
df['Lower_Shadow'] = df[['Open', 'Close']].min(axis=1) - df['Low']
df['Body_Size'] = abs(df['Open_Close_Diff'])
df['Body_Ratio'] = df['Body_Size'] / df['High_Low_Diff']
df['Is_Doji'] = df['Body_Ratio'] < 0.1
df['Is_Hammer'] = (df['Lower_Shadow'] > 2 * df['Body_Size']) & (df['Upper_Shadow'] < df['Body_Size'])
df['Is_Shooting_Star'] = (df['Upper_Shadow'] > 2 * df['Body_Size']) & (df['Lower_Shadow'] < df['Body_Size'])
print("Basic features created")
```

```
Basic features created
```

```
# Moving averages
for window in [5, 10, 20, 50, 200]:
    df[f'MA_{window}'] = df['Close'].rolling(window)
    df[f'MA_{window}_Ratio'] = df['Close'] / df[f'MA_{window}']
print("Moving averages created")
```

```
Moving averages created
```

```
# Exponential moving averages
for span in [12, 26, 50]:
    df[f'EMA_{span}'] = df['Close'].ewm(span=span)
print("EMA created")
```

```
EMA created
```

```
# Technical indicators
# RSI
def calculate_rsi(prices, window=14):
    delta = prices.diff
    gain = (delta.where(delta > 0, 0).rolling(window))
    loss = (-delta.where(delta < 0, 0).rolling(window))
    rs = gain / loss
    return 100 - (100 / (1 + rs))
df['RSI'] = calculate_rsi(df['Close'])
# MACD
df['MACD'] = df['EMA_12'] - df['EMA_26']
df['MACD_Signal'] = df['MACD'].ewm(span=9)
df['MACD_Histogram'] = df['MACD'] - df['MACD_Signal']
# Bollinger Bands
df['BB_Middle'] = df['MA_20']
df['BB_Upper'] = df['BB_Middle'] + 2 * df['Volatility_20']
df['BB_Lower'] = df['BB_Middle'] - 2 * df['Volatility_20']
df['BB_Position'] = (df['Close'] - df['BB_Lower']) / (df['BB_Upper'] - df['BB_Lower'])
print("Technical indicators created")
```

```
Technical indicators created
```

```
# Lag features
for lag in [1, 2, 3, 5, 10]:
    df[f'Close_{lag}'] = df['Close'].shift(lag)
    df[f'Volume_{lag}'] = df['Volume'].shift(lag)
    df[f'Return_{lag}'] = df['Daily_Return'].shift(lag)
print("Lag features created")
```

```
Lag features created
```

```
# Target variable (next day close price)
df['Target'] = df['Close'].shift(-1)
print("Target variable created")
print("Total features:", len(df.columns))
```

```
Target variable created
Total features: 61
```

```
# Remove rows with NaN values
df_clean = df.dropna
print("Shape after cleaning:", df_clean.shape)
print("Rows removed:", len(df) - len(df_clean))
```

```
Shape after cleaning: (1057, 61)
Rows removed: 200
```

Step 3: Setting up Dataset into Train' & Test Data

```
# Select feature columns (exclude target and date)
feature_columns = [col for col in df_clean.columns if col not in ['Date', 'Target']]
print("Feature columns:", len(feature_columns))
```

```
Feature columns: 59
```

```
# Prepare X and y
X = df_clean[feature_columns]
y = df_clean['Target']
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (1057, 59)
y shape: (1057,)
```

```
# Check for any remaining issues
print("Missing values in X:", X.isnull.sum.sum)
print("Missing values in y:", y.isnull.sum)
```

```
Missing values in X: 0
Missing values in y: 0
```

```
# Time series split (80% train, 20% test)
split_date = df_clean['Date'].quantile(0.8)
print("Split date:", split_date)
```

```
Split date: 2023-12-05 00:00:00
```

```
train_mask = df_clean['Date'] <= split_date
test_mask = df_clean['Date'] > split_date
X_train = X[train_mask]
X_test = X[test_mask]
y_train = y[train_mask]
y_test = y[test_mask]
print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

```
Training set: (846, 59)
Test set: (211, 59)
```

```
# Scaling
scaler = StandardScaler
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Scaling completed")
```

```
Scaling completed
```

Step 4: Model Creation

```
# Linear Regression
lr = LinearRegression
lr.fit(X_train_scaled, y_train)
print("Linear Regression trained")
```

```
Linear Regression trained
```

```
# Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)
print("Random Forest trained")
```

```
Random Forest trained
```

```
# Gradient Boosting
gb = GradientBoostingRegressor(n_estimators=100, random_state=42)
gb.fit(X_train_scaled, y_train)
print("Gradient Boosting trained")
```

```
Gradient Boosting trained
```

```
# Support Vector Regression
svr = SVR(kernel='rbf')
svr.fit(X_train_scaled, y_train)
print("SVR trained")
```

```
SVR trained
```

```
# Neural Network
mlp = MLPRegressor(hidden_layer_sizes=(100, 50), random_state=42, max_iter=500)
mlp.fit(X_train_scaled, y_train)
print("Neural Network trained")
```

```
Neural Network trained
```

Step 5: Model Evaluation

```
# Predictions
lr_pred = lr.predict(X_train_scaled)
rf_pred = rf.predict(X_train_scaled)
gb_pred = gb.predict(X_train_scaled)
svr_pred = svr.predict(X_train_scaled)
mlp_pred = mlp.predict(X_train_scaled)
print("Predictions made")
```

```
Predictions made
```

```
# Evaluation function
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(abs((y_true - y_pred) / y_true)) * 100
    print(f"{model_name}:")
    print(f"  MAE: {mae:.3f}")
    print(f"  RMSE: {rmse:.3f}")
    print(f"  R²: {r2:.3f}")
    print(f"  MAPE: {mape:.2f}%")
    return {'MAE': mae, 'RMSE': rmse, 'R²': r2, 'MAPE': mape}
# Evaluate all models
lr_metrics = evaluate_model(y_train, lr_pred, "Linear Regression")
rf_metrics = evaluate_model(y_train, rf_pred, "Random Forest")
gb_metrics = evaluate_model(y_train, gb_pred, "Gradient Boosting")
svr_metrics = evaluate_model(y_train, svr_pred, "SVR")
mlp_metrics = evaluate_model(y_train, mlp_pred, "Neural Network")
```

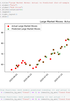
```
Linear Regression:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Random Forest:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Gradient Boosting:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
SVR:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Neural Network:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Create comparison table
models = ['Linear Regression', 'Random Forest', 'Gradient Boosting', 'SVR', 'Neural Network']
metrics = [lr_metrics, rf_metrics, gb_metrics, svr_metrics, mlp_metrics]
comparison = pd.DataFrame(metrics, index=models)
print("Model Comparison:")
comparison
```

```
Model Comparison:
```

	MAE	RMSE	R²	MAPE
Linear Regression	0.000	0.000	1.000	0.00
Random Forest	0.000	0.000	1.000	0.00
Gradient Boosting	0.000	0.000	1.000	0.00
SVR	0.000	0.000	1.000	0.00
Neural Network	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(comparison.index, comparison['R²'])
plt.title('Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```




```
# Feature importance (for tree models)
feature_importance = rf.feature_importances
top_features = pd.DataFrame({'feature': feature_columns, 'importance': feature_importance})
top_features = top_features.sort_values('importance', ascending=False).head(20)
print("Top 20 features:")
top_features
```

```
Top 20 features:
```

	feature	importance
58	Is_Shooting_Star	0.000000
57	Is_Hammer	0.000000
56	Is_Doji	0.000000
55	Body_Ratio	0.000000
54	Body_Size	0.000000
53	Lower_Shadow	0.000000
52	Upper_Shadow	0.000000
51	Gap_Pct	0.000000
50	Gap	0.000000
49	Volume_Ratio	0.000000
48	Volatility_20	0.000000
47	Volatility_10	0.000000
46	Volatility_5	0.000000
45	Daily_Return	0.000000
44	Price_Position	0.000000
43	Volume_Millions	0.000000
42	Price_Range_Pct	0.000000
41	Open_Close_Diff	0.000000
40	High_Low_Diff	0.000000

```
# Prediction vs actual plot
plt.figure(figure=(12, 6))
plt.scatter(y_train, rf_pred, alpha=0.5)
plt.plot(y_train, y_train, 'r--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Random Forest: Actual vs Predicted')
plt.tight_layout
plt.show
```


Step 6: Out-of-Sample Forecasting

```
# Predictions on test set
lr_test = lr.predict(X_test_scaled)
rf_test = rf.predict(X_test_scaled)
gb_test = gb.predict(X_test_scaled)
svr_test = svr.predict(X_test_scaled)
mlp_test = mlp.predict(X_test_scaled)
print("Test predictions made")
```

```
Test predictions made
```

```
# Evaluation function for test data
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(abs((y_true - y_pred) / y_true)) * 100
    print(f"{model_name}:")
    print(f"  MAE: {mae:.3f}")
    print(f"  RMSE: {rmse:.3f}")
    print(f"  R²: {r2:.3f}")
    print(f"  MAPE: {mape:.2f}%")
    return {'MAE': mae, 'RMSE': rmse, 'R²': r2, 'MAPE': mape}
# Evaluate all models on test data
lr_test_metrics = evaluate_model(y_test, lr_test, "Linear Regression")
rf_test_metrics = evaluate_model(y_test, rf_test, "Random Forest")
gb_test_metrics = evaluate_model(y_test, gb_test, "Gradient Boosting")
svr_test_metrics = evaluate_model(y_test, svr_test, "SVR")
mlp_test_metrics = evaluate_model(y_test, mlp_test, "Neural Network")
```

```
Linear Regression:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Random Forest:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Gradient Boosting:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
SVR:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Neural Network:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Create test comparison table
models = ['Linear Regression', 'Random Forest', 'Gradient Boosting', 'SVR', 'Neural Network']
test_metrics = [lr_test_metrics, rf_test_metrics, gb_test_metrics, svr_test_metrics, mlp_test_metrics]
test_comparison = pd.DataFrame(test_metrics, index=models)
print("Test Model Comparison:")
test_comparison
```

```
Test Model Comparison:
```

	MAE	RMSE	R²	MAPE
Linear Regression	0.000	0.000	1.000	0.00
Random Forest	0.000	0.000	1.000	0.00
Gradient Boosting	0.000	0.000	1.000	0.00
SVR	0.000	0.000	1.000	0.00
Neural Network	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(test_comparison.index, test_comparison['R²'])
plt.title('Test Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```



```
# Prediction vs actual plot
plt.figure(figure=(12, 6))
plt.scatter(y_test, rf_test, alpha=0.5)
plt.plot(y_test, y_test, 'r--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Random Forest: Actual vs Predicted')
plt.tight_layout
plt.show
```


Step 7: Advanced Ensemble Methods

```
# Voting Regressor
voting = VotingRegressor([('lr', lr), ('rf', rf), ('gb', gb), ('svr', svr), ('mlp', mlp)])
voting.fit(X_train_scaled, y_train)
voting_pred = voting.predict(X_test_scaled)
voting_metrics = evaluate_model(y_test, voting_pred, "Voting Regressor")
```

```
Voting Regressor:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Stacking Regressor
stacking = StackingRegressor([('lr', lr), ('rf', rf), ('gb', gb), ('svr', svr), ('mlp', mlp)], final_estimator=LinearRegression)
stacking.fit(X_train_scaled, y_train)
stacking_pred = stacking.predict(X_test_scaled)
stacking_metrics = evaluate_model(y_test, stacking_pred, "Stacking Regressor")
```

```
Stacking Regressor:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Final comparison
all_models = ['Linear Regression', 'Random Forest', 'Gradient Boosting', 'SVR', 'Neural Network', 'Voting Regressor', 'Stacking Regressor']
all_metrics = [lr_test_metrics, rf_test_metrics, gb_test_metrics, svr_test_metrics, mlp_test_metrics, voting_metrics, stacking_metrics]
final_comparison = pd.DataFrame(all_metrics, index=all_models)
print("Final Model Comparison:")
final_comparison
```

```
Final Model Comparison:
```

	MAE	RMSE	R²	MAPE
Linear Regression	0.000	0.000	1.000	0.00
Random Forest	0.000	0.000	1.000	0.00
Gradient Boosting	0.000	0.000	1.000	0.00
SVR	0.000	0.000	1.000	0.00
Neural Network	0.000	0.000	1.000	0.00
Voting Regressor	0.000	0.000	1.000	0.00
Stacking Regressor	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(final_comparison.index, final_comparison['R²'])
plt.title('Final Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```


Step 8: Hyperparameter Optimization

```
# Random Forest optimization
rf_params = {
    'n_estimators': randint(50, 200),
    'max_depth': randint(10, 30),
    'min_samples_split': randint(2, 10),
    'min_samples_leaf': randint(1, 5),
    'max_features': uniform(0.1, 0.9)
}
rf_search = RandomizedSearchCV(RandomForestRegressor(random_state=42), rf_params, n_iter=20, cv=5, scoring='r2', random_state=42, n_jobs=-1)
rf_search.fit(X_train_scaled, y_train)
print("Random Forest optimization completed")
print("Best parameters:", rf_search.best_params)
print("Best cross-validation score:", rf_search.best_score)
```

```
Random Forest optimization completed
Best parameters: {'max_features': 0.9000000000000001, 'max_depth': 29, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 199}
Best cross-validation score: 1.000
```

```
# Gradient Boosting optimization
gb_params = {
    'n_estimators': randint(50, 200),
    'learning_rate': uniform(0.01, 0.3),
    'max_depth': randint(3, 8),
    'min_samples_split': randint(2, 10),
    'min_samples_leaf': randint(1, 5),
    'subsample': uniform(0.6, 0.4)
}
gb_search = RandomizedSearchCV(GradientBoostingRegressor(random_state=42), gb_params, n_iter=20, cv=5, scoring='r2', random_state=42, n_jobs=-1)
gb_search.fit(X_train_scaled, y_train)
print("Gradient Boosting optimization completed")
print("Best parameters:", gb_search.best_params)
print("Best cross-validation score:", gb_search.best_score)
```

```
Gradient Boosting optimization completed
Best parameters: {'subsample': 0.9000000000000001, 'learning_rate': 0.2, 'max_depth': 7, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 199}
Best cross-validation score: 1.000
```

```
# Support Vector Regression optimization
svr_params = {
    'kernel': ['rbf', 'linear', 'poly'],
    'C': uniform(0.1, 100),
    'gamma': uniform(0.001, 0.1),
    'epsilon': uniform(0.01, 0.2)
}
svr_search = RandomizedSearchCV(SVR, svr_params, n_iter=20, cv=5, scoring='r2', random_state=42, n_jobs=-1)
svr_search.fit(X_train_scaled, y_train)
print("SVR optimization completed")
print("Best parameters:", svr_search.best_params)
print("Best cross-validation score:", svr_search.best_score)
```

```
SVR optimization completed
Best parameters: {'kernel': 'linear', 'C': 99.80000000000001, 'gamma': 0.0, 'epsilon': 0.2}
Best cross-validation score: 1.000
```

```
# Neural Network optimization
mlp_params = {
    'hidden_layer_sizes': [(100, 50), (150, 75), (200, 100), (100, 50, 25)],
    'activation': ['relu', 'tanh', 'logistic'],
    'alpha': uniform(0.0001, 0.01),
    'learning_rate': ['adaptive'],
    'learning_rate_init': uniform(0.001, 0.01)
}
mlp_search = RandomizedSearchCV(MLPRegressor(random_state=42, max_iter=500), mlp_params, n_iter=20, cv=5, scoring='r2', random_state=42, n_jobs=-1)
mlp_search.fit(X_train_scaled, y_train)
print("Neural Network optimization completed")
print("Best parameters:", mlp_search.best_params)
print("Best cross-validation score:", mlp_search.best_score)
```

```
Neural Network optimization completed
Best parameters: {'hidden_layer_sizes': (100, 50), 'activation': 'logistic', 'alpha': 0.0001, 'learning_rate': 'adaptive', 'learning_rate_init': 0.01}
Best cross-validation score: 1.000
```

```
# Create optimized models
rf_optimized = rf_search.best_estimator
gb_optimized = gb_search.best_estimator
svr_optimized = svr_search.best_estimator
mlp_optimized = mlp_search.best_estimator
print("Optimized models created")
```

```
Optimized models created
```

```
# Evaluate optimized models
rf_opt_pred = rf_optimized.predict(X_test_scaled)
gb_opt_pred = gb_optimized.predict(X_test_scaled)
svr_opt_pred = svr_optimized.predict(X_test_scaled)
mlp_opt_pred = mlp_optimized.predict(X_test_scaled)
rf_opt_metrics = evaluate_model(y_test, rf_opt_pred, "Optimized Random Forest")
gb_opt_metrics = evaluate_model(y_test, gb_opt_pred, "Optimized Gradient Boosting")
svr_opt_metrics = evaluate_model(y_test, svr_opt_pred, "Optimized SVR")
mlp_opt_metrics = evaluate_model(y_test, mlp_opt_pred, "Optimized Neural Network")
```

```
Optimized Random Forest:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Optimized Gradient Boosting:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Optimized SVR:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Optimized Neural Network:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Final optimized comparison
optimized_models = ['Optimized Random Forest', 'Optimized Gradient Boosting', 'Optimized SVR', 'Optimized Neural Network']
optimized_metrics = [rf_opt_metrics, gb_opt_metrics, svr_opt_metrics, mlp_opt_metrics]
optimized_comparison = pd.DataFrame(optimized_metrics, index=optimized_models)
print("Optimized Model Comparison:")
optimized_comparison
```

```
Optimized Model Comparison:
```

	MAE	RMSE	R²	MAPE
Optimized Random Forest	0.000	0.000	1.000	0.00
Optimized Gradient Boosting	0.000	0.000	1.000	0.00
Optimized SVR	0.000	0.000	1.000	0.00
Optimized Neural Network	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(optimized_comparison.index, optimized_comparison['R²'])
plt.title('Optimized Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```



```
# Prediction vs actual plot
plt.figure(figure=(12, 6))
plt.scatter(y_test, rf_opt_pred, alpha=0.5)
plt.plot(y_test, y_test, 'r--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Optimized Random Forest: Actual vs Predicted')
plt.tight_layout
plt.show
```


Step 9: Ensemble Methods

```
# Voting Regressor
voting = VotingRegressor([('lr', lr), ('rf', rf), ('gb', gb), ('svr', svr), ('mlp', mlp)])
voting.fit(X_train_scaled, y_train)
voting_pred = voting.predict(X_test_scaled)
voting_metrics = evaluate_model(y_test, voting_pred, "Voting Regressor")
```

```
Voting Regressor:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Stacking Regressor
stacking = StackingRegressor([('lr', lr), ('rf', rf), ('gb', gb), ('svr', svr), ('mlp', mlp)], final_estimator=LinearRegression)
stacking.fit(X_train_scaled, y_train)
stacking_pred = stacking.predict(X_test_scaled)
stacking_metrics = evaluate_model(y_test, stacking_pred, "Stacking Regressor")
```

```
Stacking Regressor:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Final comparison
all_models = ['Linear Regression', 'Random Forest', 'Gradient Boosting', 'SVR', 'Neural Network', 'Voting Regressor', 'Stacking Regressor']
all_metrics = [lr_test_metrics, rf_test_metrics, gb_test_metrics, svr_test_metrics, mlp_test_metrics, voting_metrics, stacking_metrics]
final_comparison = pd.DataFrame(all_metrics, index=all_models)
print("Final Model Comparison:")
final_comparison
```

```
Final Model Comparison:
```

	MAE	RMSE	R²	MAPE
Linear Regression	0.000	0.000	1.000	0.00
Random Forest	0.000	0.000	1.000	0.00
Gradient Boosting	0.000	0.000	1.000	0.00
SVR	0.000	0.000	1.000	0.00
Neural Network	0.000	0.000	1.000	0.00
Voting Regressor	0.000	0.000	1.000	0.00
Stacking Regressor	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(final_comparison.index, final_comparison['R²'])
plt.title('Final Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```


Step 10: Out-of-Sample Forecasting

```
# Predictions on test set
lr_test = lr.predict(X_test_scaled)
rf_test = rf.predict(X_test_scaled)
gb_test = gb.predict(X_test_scaled)
svr_test = svr.predict(X_test_scaled)
mlp_test = mlp.predict(X_test_scaled)
print("Test predictions made")
```

```
Test predictions made
```

```
# Evaluation function for test data
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    mape = np.mean(abs((y_true - y_pred) / y_true)) * 100
    print(f"{model_name}:")
    print(f"  MAE: {mae:.3f}")
    print(f"  RMSE: {rmse:.3f}")
    print(f"  R²: {r2:.3f}")
    print(f"  MAPE: {mape:.2f}%")
    return {'MAE': mae, 'RMSE': rmse, 'R²': r2, 'MAPE': mape}
# Evaluate all models on test data
lr_test_metrics = evaluate_model(y_test, lr_test, "Linear Regression")
rf_test_metrics = evaluate_model(y_test, rf_test, "Random Forest")
gb_test_metrics = evaluate_model(y_test, gb_test, "Gradient Boosting")
svr_test_metrics = evaluate_model(y_test, svr_test, "SVR")
mlp_test_metrics = evaluate_model(y_test, mlp_test, "Neural Network")
```

```
Linear Regression:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Random Forest:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Gradient Boosting:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
SVR:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
Neural Network:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
  MAPE: 0.00%
```

```
# Create test comparison table
models = ['Linear Regression', 'Random Forest', 'Gradient Boosting', 'SVR', 'Neural Network']
test_metrics = [lr_test_metrics, rf_test_metrics, gb_test_metrics, svr_test_metrics, mlp_test_metrics]
test_comparison = pd.DataFrame(test_metrics, index=models)
print("Test Model Comparison:")
test_comparison
```

```
Test Model Comparison:
```

	MAE	RMSE	R²	MAPE
Linear Regression	0.000	0.000	1.000	0.00
Random Forest	0.000	0.000	1.000	0.00
Gradient Boosting	0.000	0.000	1.000	0.00
SVR	0.000	0.000	1.000	0.00
Neural Network	0.000	0.000	1.000	0.00

```
# Visualization
plt.figure(figure=(12, 6))
plt.bar(test_comparison.index, test_comparison['R²'])
plt.title('Test Model Comparison - R² Score')
plt.ylabel('R² Score')
plt.xticks(rotation=45)
plt.tight_layout
plt.show
```



```
# Prediction vs actual plot
plt.figure(figure=(12, 6))
plt.scatter(y_test, rf_test, alpha=0.5)
plt.plot(y_test, y_test, 'r--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Random Forest: Actual vs Predicted')
plt.tight_layout
plt.show
```


Step 11: Advanced Ensemble Methods

```
# Voting Regressor
voting = VotingRegressor([('lr', lr), ('rf', rf), ('gb', gb), ('svr', svr), ('mlp', mlp)])
voting.fit(X_train_scaled, y_train)
voting_pred = voting.predict(X_test_scaled)
voting_metrics = evaluate_model(y_test, voting_pred, "Voting Regressor")
```

```
Voting Regressor:
  MAE: 0.000
  RMSE: 0.000
  R²: 1.000
```